REVATURE

Revature Expense Manager – Phase 3

Brian Paul
Scot Putnam
Matthew Washburn
Landon Wu

Tech Stack

Core technologies powering development and automated deployment

Docker

Containerization

Containerization of both Python and Java applications to guarantee environment consistency.

Jenkins

Orchestration

Drives automated CI/CD pipeline orchestration, testing, and delivery.

AWS

Cloud Infrastructure

Cloud hosting for the Jenkins controller and all target deployment platforms.

Git

Source Control

Version control system and the trigger mechanism for automated pipeline runs.

REVATURE

App Setup

Environment Flexible Frontend

One frontend route contract now works in both development and production.

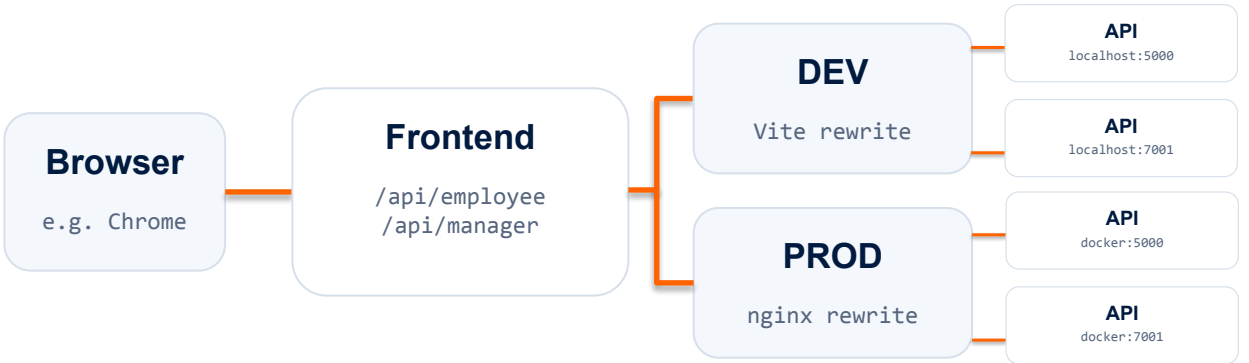
WHAT CHANGED

- *config.js* - Relative constants keep backend hosts and ports out of the compiled frontend bundle.

```
export const EMPLOYEE_API = "/api/employee"
export const MANAGER_API = "/api/manager"
```

- *vite.config.js* & *nginx.config* - both wire /api/employee and /api/manager to the designated API routes.
  - *vite* = local development
  - *nginx* = production reverse-proxy

ONE LOGICAL ROUTE MAP



CODE `frontend/src/config.js` • `frontend/vite.config.js` • `frontend/nginx.conf`

Health & CORS - Runtime Contracts

WHAT CHANGED

- Flask and Javalin each expose an unauthenticated GET /health endpoint.

  Am I alive?

- Both apps read CORS_ORIGINS to recognize an allowed frontend origin.

  Who may call me?

CODE `employee_app/main.py` • `manager-app/.../AppFactory.java`

Postgres Integration

1 - RUNTIME STATE

Postgres 17

```
postgres-data
named volume
```

- SQL network service with persistent volume and concurrent writes, rather than a single SQLite file.

2 - APP CONNECTIONS

Python + Java

```
psycopg pool
JDBC + HikariCP
```

- Python and Java both moved to pooled, environment-resolved database access.

3 - TEST CASCADE

Testing DB

```
pytest conftest.py
Java TestPostgres
```

- Pytest uses a Postgres Testcontainer that uses TRUNCATE and rebuilding.
- Java creates and points to new per-class Postgres databases in a shared container that get closed after use.

4 - BENEFIT

Parity + isolation + network capability

```
same Postgres engine
repeatable clean state
```

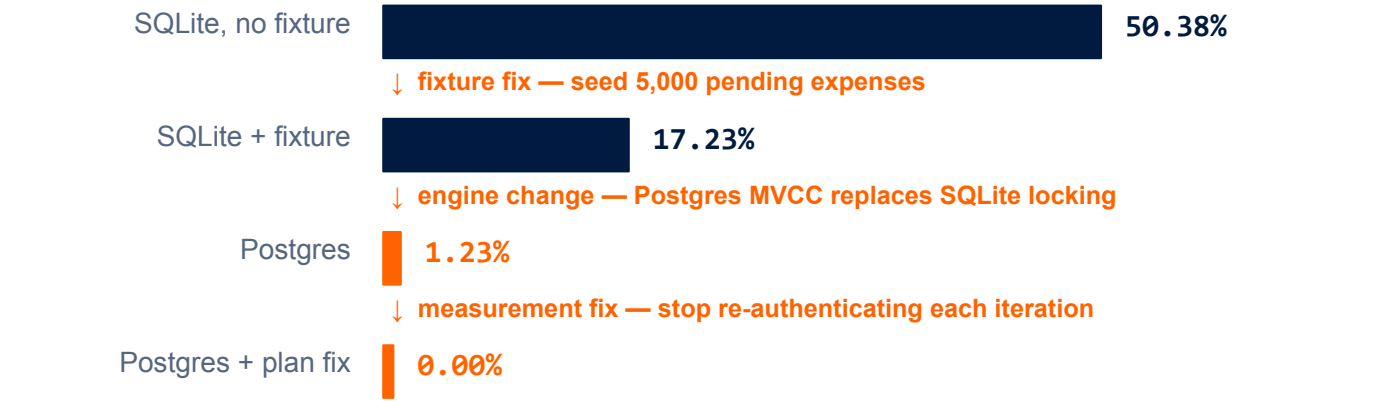
- Parity between app runtime and test suite, plus recovery from the single-writer bottleneck of SQLite.

Jmeter Conversion - Load Testing for CI

WHAT CHANGED

- The first defect was the fixture — ids 1–5,000 reviewed against 8 seeded rows measured the rejection path.
- Seeding 5,000 expenses and 5,000 approvals in one batched transaction made every sample a real write.
- Login once per thread removed 4,900 bcrypt logins; write p95 fell 2,077 ms to 5 ms with no app change.

SUITE FAILURE RATE	SAMPLES	STAGE DURATION	WRITE P95	CI GATE
0.00%	6,800	02:17	5 ms	2%
from 17.23%	from 11,700	from 03:15	from 3781 ms	from 21%



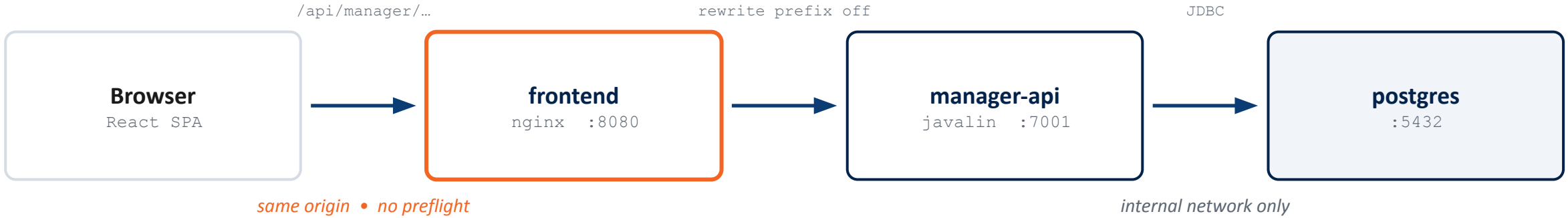
CODE `database/schema.sql` • `DatabaseSeeder.java` • `manager_write_performance_test.jmx`

REVATURE

Docker Containerization

Networking: One Origin, One Way In

Every call leaves the browser on the same port the page came from



nginx reverse proxy: one origin, no CORS problem

```
/api/employee/*   →   employee-api:5000
/api/manager/*    →   manager-api:7001
```

The page and every API call share one origin, so the browser never makes a cross-origin request and never sends a preflight. nginx rewrites the prefix off and forwards on the internal network.

Service name = hostname

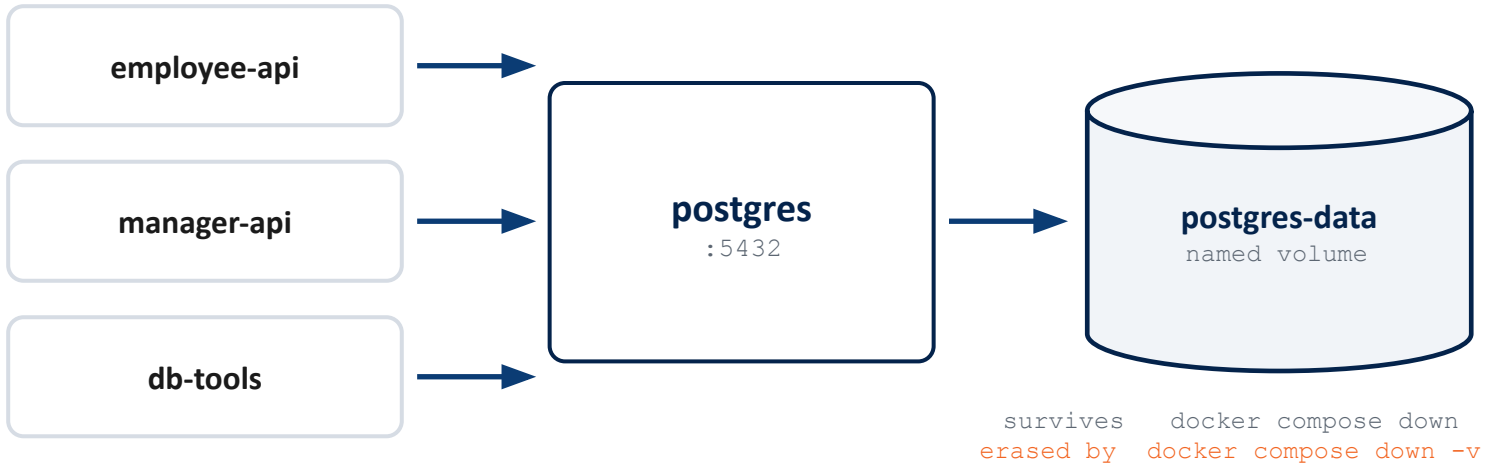
Compose builds one bridge network. Docker's DNS at 127.0.0.11 resolves manager-api to a container IP, no IP addresses in our config.

Production publishes one port

Only the frontend's :80 is exposed. Both APIs and Postgres have no host port at all, the proxy is the only door.

Data: The Volume and the Sidecar

Containers are disposable — the volume is not



`db-init` `profiles: ["init"]`
One-shot. It drops and rebuilds every table, so it can never run on a normal up.

Phase 2 shared one SQLite file across four mounts. Now the services hold connections, not file handles.

`db-tools`

The E2E reset sidecar

0.3s

per scenario, via compose exec

Same image as db-init, left running on sleep infinity. A live container to exec into costs 0.3s; spinning up a fresh one with run --rm costs about 2s — every scenario, every build.

Three Compose Files, Three Environments

Same four services everywhere — what changes is where the images come from

`compose.yaml`
Dev box

- Builds every image from source
- Ports 5000 / 7001 / 8080 published so you can hit an API directly
- Postgres on 5433 — 5432 was already taken on a dev machine

+

`+ compose.ci.yaml`
Jenkins agent

- Layered on the dev file, not a copy of it
- Ports shifted to 15000 / 17001 / 18080 so CI never fights a dev stack
- Adds health checks so up --wait actually means ready

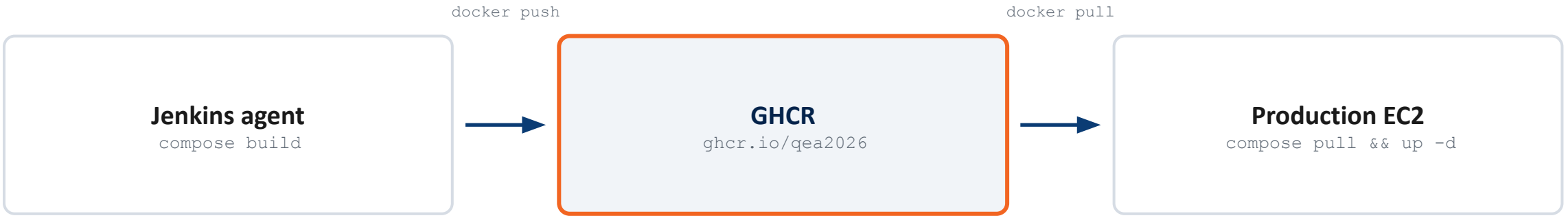
`compose.prod.yaml`
Production EC2

- Standalone — no build section, because there is no source tree there
- Pulls images by commit tag
- Only :80 published

Why CI layers instead of copying: its job is to test the stack developers actually run. A second full file would be free to drift.

Shipping the Images: GHCR

A registry is npm for container images — build once, run the identical bytes in production




```
ghcr.io/qea2026/expense-frontend : a1b2c3d
```
registry + org · image name · tag

Four images ship every build: expense-frontend, expense-employee-api, expense-manager-api, expense-db-init.

The name decides the destination

Compose names images for the registry at build time, so publishing is a push — never a retag.

GHCR over Docker Hub

Same org, same permissions, same login as the repo. No second account to manage.

Production-ready images: manager-api is a three-stage build with a jlink-trimmed runtime — about 61 MB instead of 90 MB.

REVATURE

Jenkins Pipeline

Twelve Stages, Three Phases

Cheapest checks first, deployment last

VERIFY

Before anything is built

1 Clean reports

2 Test database

3 Unit + Integration - Java + Python

INTEGRATE

A real stack, exercised end to end

4 Build images

5 Seed database

6 Stack up

7 Smoke test

8 E2E

9 Load test

RELEASE

Only once the gates above are green

10 Publish images

11 Deploy

12 Verify deployment

Fail fast: the unit and DAO suites run before a single image is built, so a broken commit never reaches the registry.

Jenkins Best Practices

PIPELINE DESIGN

- ✓ Declarative syntax
- ✓ Jenkinsfile in source control
- ✓ Fail fast - tests before images
- ✓ Parallel where independent
- ✓ Meaningful stage names
- ✓ Conditional stages via when

SECURITY

- ✓ Credential store, nothing hardcoded
- ✓ Per-stage credential binding
- ✓ Secrets over stdin, never argv
- ✓ HTTPS behind a reverse proxy
- ✓ Sign-on via GitHub OAuth
- ✓ Auth matrix, only needed privileges
- ✓ Controller has no Docker socket

PERFORMANCE & STORAGE

- ✓ Builds run on an agent
- ✓ Parallel test suites
- ✓ Build rotation - keep last 20
- ✓ Image tags are named after commits

What We Would Add Next

Static analysis	No SonarQube or Checkstyle gate today. It would run parallel to the existing tests essentially for free
`options { timeout }`	Nothing bounds a hung stage. A wedged Selenium run holds the agent until a human notices.
JCasC - Jenkins Config as Code	The controller is hand-configured. Rebuilding it means redoing OAuth, credentials and tool definitions by hand, from memory.
`JENKINS_HOME backup`	One named volume, one EC2 instance, no snapshot, corruption/loss would require a full rebuild

REVATURE

CI/CD Architecture

CI/CD Architecture – Improvements

IAM Roles

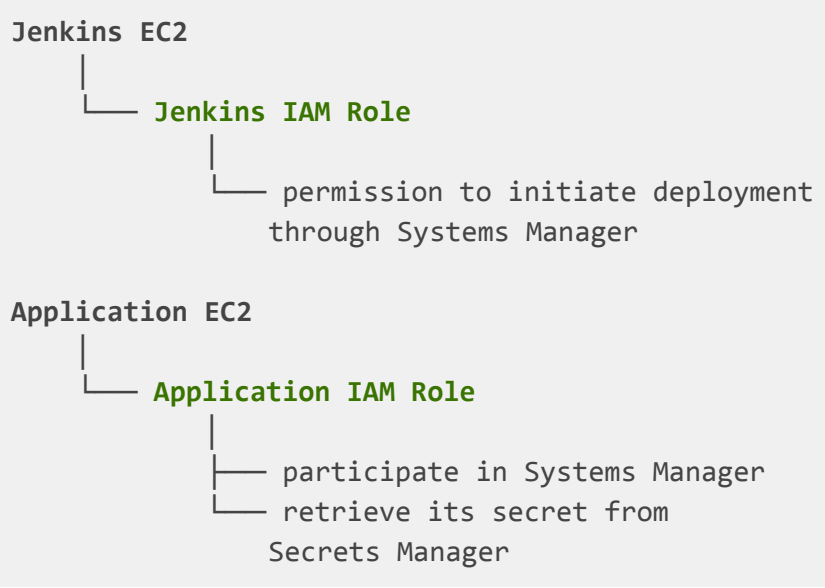
- Assign separate roles to Jenkins EC2 and Application EC2
- Grant only required AWS permissions using least privilege
- Use temporary AWS credentials instead of stored access keys

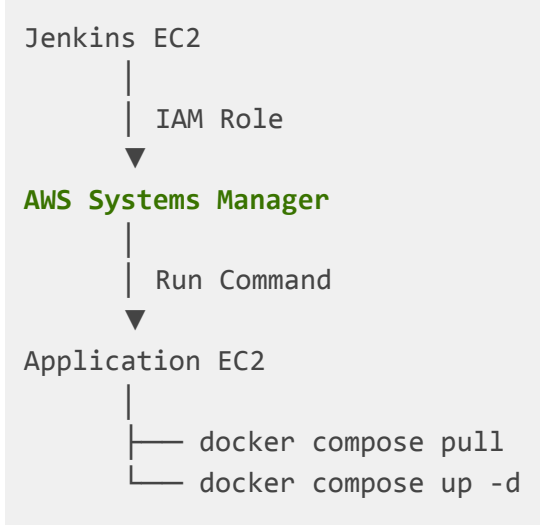
AWS Systems Manager (SSM)

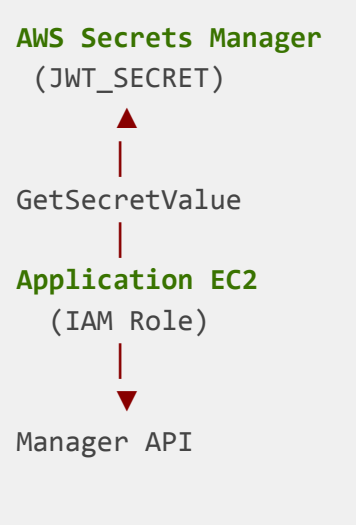
- Replace direct SSH/SCP deployment with AWS-managed remote execution
- Removes Jenkins deployment SSH key
- Eliminate inbound SSH port 22 for deployment

AWS Secrets Manager

- Move production `JWT_SECRET` out of the EC2 `.env` file
- Application EC2 retrieves secrets using its IAM role
- Centralize encrypted secret storage and access control







REVATURE

Retrospectives

Thank you!